A. D. PATEL INSTITUTE OF TECHNOLOGY
Department of Information Technology

P R O J E C T R E P O R T

Travel Planner

Using Python, Tkinter, Dictionaries, File Handling and Exception Handling

Project Type	Python Desktop Application / Mini Project
Subject	Programming With Python
Technologies	Python 3, Tkinter, pickle, csv
Core Concept	Itinerary, budget and file handling using dictionaries, lists and exceptions
Prepared By	Aryan Joshi - 12402080601008
Department	Information Technology
Institute	A. D. Patel Institute of Technology, CVM University

Table of Contents

1. Abstract....3
2. Introduction....3
3. Problem Statement....3
4. Objectives....3
5. Scope of the Project....3
6. Tools and Technologies....4
7. System Requirements....4
8. System Design and Working....4
9. Implementation Details....6
10. Data Structures and Budget Logic....7
11. Testing....9
12. Results and Discussion....9
13. Advantages....10
14. Limitations and Security Considerations....10
15. Future Enhancements....10
16. Conclusion....10
17. References....11

1. Abstract

This report describes Travel Planner, a Python desktop application for planning a trip. A traveller can build an itinerary from destinations, record bookings and expenses, check the total cost against a budget limit, and save the whole plan to a file. The window is built with Tkinter. The entire trip is held in one dictionary called the plan, which contains the trip name, the budget limit, the preferences and three lists of dictionaries for destinations, bookings and expenses. Functions that calculate the budget and build the itinerary text contain no Tkinter code, so they are independent of the interface. Plans are saved and loaded with `pickle` and exported with the `csv` module and as text files. A small set of custom exceptions, handled with `try`, `except`, `else` and `finally`, turns wrong input and bad files into friendly messages instead of crashes. The project combines dictionaries, lists, functions, file handling, exception handling and GUI programming in one working application.

2. Introduction

Planning a trip involves many small pieces of information: where to go, how many days to stay, what each day costs, which tickets and hotels are booked and what has been spent so far. When a group plans together, these details are often scattered. One person keeps hotel details in a chat, another keeps costs in a notebook, and nobody knows the total until the money has run out.

Travel Planner puts all of this in one place. The user enters destinations with the number of days and the cost per day, adds bookings and expenses, and chooses preferences such as the trip style, the mode of transport, a hotel and insurance. The program produces a day-wise itinerary, adds up the cost and compares it with the budget limit. It works offline, needs no account, and stores the plan in a local file so that it can be reopened later. This makes it suitable for students, families, solo travellers and groups.

3. Problem Statement

Trip details are scattered across chats and notes, costs are hard to track and budgets are easily exceeded, and plans made in a program disappear when the program is closed. A simple application is needed that keeps destinations, bookings, expenses and preferences together, calculates the budget automatically, warns when the limit is crossed, and saves the plan to a file so that it can be loaded again. It should run offline on an ordinary computer and should report mistakes in the input politely instead of stopping.

4. Objectives

The objectives of the project are:

- to apply dictionaries, lists of dictionaries, functions, file handling and exception handling from the syllabus in one project;
- to build a friendly Tkinter interface with forms, lists, tabs and menus;
- to create a day-wise itinerary from destinations, days and daily cost, for example Day 1-3: Goa;
- to calculate the total cost from destination estimates, active bookings and expenses, and warn when it exceeds the limit;
- to save and load the plan across sessions, and to export it to CSV and text files;
- to validate all input and handle errors so that the program shows a message instead of crashing;
- to keep the calculation logic separate from the graphical interface.

5. Scope of the Project

The application is a single-user, offline tool that works with local files. It is meant for learning and for planning ordinary trips.

The current version does:

- store the trip name, budget limit and preferences;
- manage lists of destinations, bookings and expenses;
- build an itinerary and a budget report;
- save, load and export the plan.

The current version does not:

- book tickets or hotels online, or show live prices;
- show maps or weather, or convert currencies;
- share a plan between several users or store it in the cloud;
- export to PDF or use a database.

6. Tools and Technologies

- **Python 3:** the language for the data structures, functions and program logic.
- **Tkinter and ttk:** build the window, menu bar, forms, tabs and lists. The widgets used are Entry, Combobox, Radiobutton, Checkbutton, Listbox, Text, Notebook and Scrollbar.
- **Dictionaries and lists:** hold the plan and its destinations, bookings and expenses.
- **pickle module:** saves and loads the complete plan as a binary `.dat` file.
- **csv module:** exports the destinations in a form that opens in a spreadsheet.
- **Exception handling:** custom exception classes with `try`, `except`, `else` and `finally`.
- **Text editor or IDE:** any editor can be used to write the code; the program is started from a terminal or an IDE.

7. System Requirements

Hardware requirements

A desktop or laptop computer with a graphical desktop. The program needs very little memory and disk space.

Software requirements

Python 3 with Tkinter, which is included in the standard Windows installer for Python. Only modules from the Python standard library are used, so nothing else has to be installed. The program is started with `python travel_planner.py`.

8. System Design and Working

The application is divided into three layers, shown in Figure 1. The interface layer is the `TravelPlannerApp` class, which builds the Tkinter window and reacts to buttons and menus. The logic layer holds ordinary functions such as `calculate_budget()` and `build_itinerary_text()`. The storage layer contains the functions that save, load and export the plan. The plan dictionary is the single source of truth: the screen is always redrawn from it, and every layer reads or changes the same dictionary.

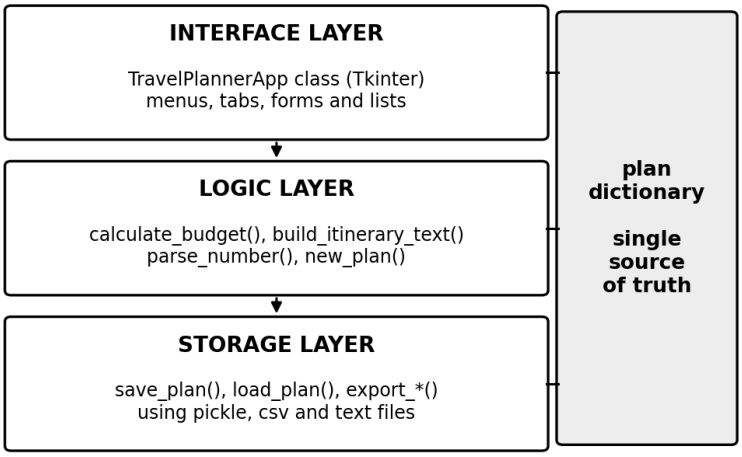


Figure 1: The three layers of Travel Planner

Figure 2 follows one action from start to finish. The user fills in a form and clicks a button. The input is read and converted to a number where needed. If it is valid, it is appended to the right list in the plan and `refresh()` redraws the lists on the screen; if it is not valid, an exception is raised and a warning box is shown. Later the user can click Calculate Budget, which asks the logic layer for a summary, or choose File > Save, which writes the plan to disk.

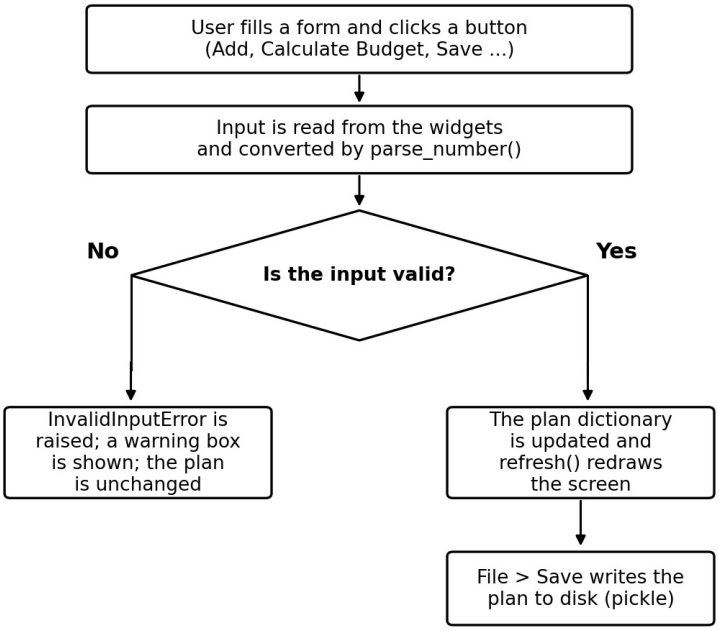


Figure 2: Flow of one user action through the program

Two design decisions are worth noting.

- The calculation functions take the plan as an argument and return a result. They contain no Tkinter code, so they can be tested without opening a window.
- The Listboxes are never edited directly. Every change is made to the plan first and the screen is then redrawn by one method, `refresh()`. This keeps the screen and the data in step.

9. Implementation Details

9.1 The plan dictionary

`new_plan()` creates an empty plan. The plan is a dictionary whose values are numbers, strings, a nested dictionary and three lists. Each destination, booking and expense is itself a small dictionary, so every item has named fields and the order of the items is kept. Listing 1 shows the structure and Table 1 describes the parts.

```
plan = {
    "trip_name": "Goa Trip",
    "budget_limit": 20000.0,
    "preferences": {...},
    "destinations": [ {...} ],
    "bookings": [ {...} ],
    "expenses": [ {...} ]
}
```

Listing 1: Structure of the plan dictionary

Table 1: Parts of the plan dictionary

Key	Type	Contents
`trip_name`	string	Name of the trip, for example Goa Trip
`budget_limit`	number	The most the traveller wants to spend
`preferences`	dictionary	Trip style, mode of transport, hotel and insurance choices
`destinations`	list of dictionaries	Place, number of days and cost per day
`bookings`	list of dictionaries	Booking type, cost and whether it is active or cancelled
`expenses`	list of dictionaries	Money already spent, with a description and an amount

9.2 The logic functions

`parse_number()` converts the text typed by the user into a number. It raises `InvalidInputError` when the text is empty, contains letters or is negative. `calculate_budget(plan)` adds the destination estimates, the active bookings and the expenses and returns a summary dictionary. `build_itinerary_text(plan)` goes through the destinations in order and produces a day-wise text such as Day 1-3: Goa. None of these functions uses Tkinter.

9.3 File handling

`save_plan()` writes the plan dictionary to a `.dat` file with `pickle.dump`, and `load_plan()` reads it back with `pickle.load`. Both open the file in a `with` block, so the file is always closed, even if an error occurs. After loading, the data is checked before it is used, and a file that does not have the expected form raises `DataFormatError`. The export functions write the destinations to a `.csv` file with the `csv` module, and the itinerary and budget report to a `.txt` file.

> **Note:** A pickle file can run code when it is loaded. A plan file should therefore be opened only if it was created by this application. Section 15 suggests JSON as a safer format for a future version.

9.4 Error handling

All errors of the application come from one base class, `TravelPlannerError`, so the interface can catch them in one place and show a message box. The two classes below it describe the two main kinds

of failure (Table 2). In the code, `try` holds the risky statement, `except` handles the known errors, `else` runs only when nothing went wrong, for example when the destination is added to the list, and `finally` runs in every case.

Table 2: Exception classes

Exception	Raised when	What the user sees
`TravelPlannerError`	Base class of all application errors	A message box (caught in one place)
`InvalidInputError`	A field is empty, has letters or is negative	A warning; nothing is added
`DataFormatError`	A file is corrupt or has the wrong format	A clear error message

9.5 The graphical interface

The window has a menu bar with File, Export and Help, a panel for the trip preferences, a tabbed work area and a status bar. The tabs are Destinations, Bookings, Expenses and Budget. The three data tabs share one reusable method, `build_tab`, which creates the form and the Listbox with its scroll bar, so the code is written only once. Table 3 lists the widgets and what each is used for.

Table 3: Tkinter widgets and their use

Widget	Use in the application
`Entry`	Typed values such as the trip name, destination, days and cost
`Combobox`	Fixed choices such as the trip style and the booking type
`Radiobutton`	One choice out of several, such as the mode of transport
`Checkbutton`	Yes or no options such as hotel and insurance
`Listbox with Scrollbar`	The current destinations, bookings and expenses
`Text`	The itinerary and the budget report
`Notebook`	The tabs that keep the interface uncluttered

10. Data Structures and Budget Logic

The program uses a few simple structures, each for a clear purpose. A dictionary keeps the named parts of the plan. A list of dictionaries keeps the items of each kind in order, which matters for the itinerary. Functions take the plan as an argument and return summaries, so the same data can be used by the screen, by the file functions and by the exports.

The total cost of the trip is the sum of three parts. Listing 2 shows the idea in pseudocode.

```
CALCULATE_BUDGET(plan):
    estimates = sum of (days x cost per day) for each destination
    bookings  = sum of the cost of each booking that is Active
    expenses  = sum of the amount of each expense
    total     = estimates + bookings + expenses
    remaining = budget_limit - total
    over_limit = total > budget_limit
    return summary
```

Listing 2: Budget calculation (simplified pseudocode)

Cancelled bookings are left out of the total, so cancelling a booking immediately lowers the cost. As an illustration, take a trip with a limit of Rs. 20,000. It has two destinations, Goa for 3 days at Rs. 2,000 a day and Lonavala for 2 days at Rs. 1,500 a day. It has a train ticket of Rs. 2,400 and a hotel of Rs. 4,000, both active, and a bus ticket of Rs. 800 that was cancelled. Food and souvenirs have cost Rs. 2,100 so far. Table 4 and Figure 3 show the result. These numbers are an example to explain the calculation and not output of the program.

Table 4: Worked budget example

Part	Calculation	Amount (Rs.)
Destination estimates	3 x 2,000 + 2 x 1,500	9,000
Active bookings	2,400 + 4,000 (cancelled 800 left out)	6,400
Expenses	Food and souvenirs	2,100
Total	9,000 + 6,400 + 2,100	17,500
Remaining	20,000 - 17,500	2,500

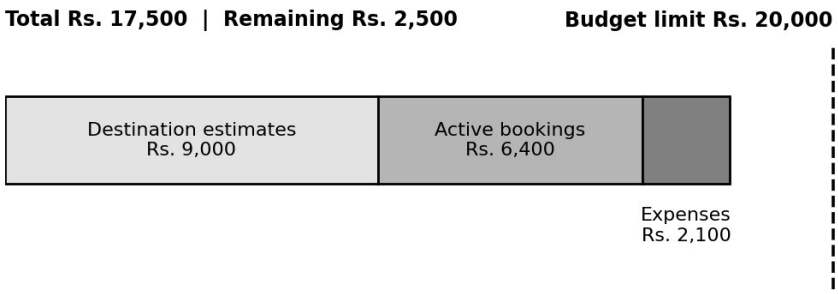


Figure 3: The example trip against its budget limit

11. Testing

The application was tested with the scenarios in Table 5. They cover wrong input, the budget rule for cancelled bookings, saving and reloading, and bad files. The last column gives the behaviour the program is designed to show in each case.

Table 5: Test scenarios and expected behaviour

No.	Scenario	Input or action	Expected result
1	Empty field	Leave name or days empty and click Add	A warning is shown; nothing is added
2	Letters in a number field	Type abc as the number of days	InvalidInputError; a warning box appears
3	Negative number	Type -5 as the cost	A warning is shown; nothing is added
4	Valid destination	Name, 3 days, cost per day	Added to the list; itinerary shows the days
5	Cancelled booking	Set a booking to cancelled	It is left out of the budget total
6	Over the limit	Total higher than the limit	A warning is shown with the budget
7	Save, close, reopen	File > Save, restart, File > Open	The data is identical to before
8	Missing file	Open a file that does not exist	A File not found message
9	Corrupt or wrong file	Open a file that is not a saved plan	DataFormatError; a clear message

Points observed during testing:

- Every wrong input ends in a message box, never in a crash.
- The main difficulty was keeping the Listboxes consistent with the data. It was solved by redrawing them from the plan dictionary through the single `refresh()` method.
- Because the logic functions have no Tkinter code, they can be checked with ordinary calls and a sample plan.

12. Results and Discussion

The program brings the whole trip into one window. The user can build an itinerary, add bookings and expenses, see the total and the remaining budget, and keep the plan for later. The budget is calculated automatically, which removes the need to add up amounts by hand and shows at once when the limit is crossed. Saving and loading with `pickle` keeps the plan between sessions, and the CSV and text exports allow it to be shared or printed.

Error handling makes the program dependable. Because all application errors share one base class, the interface needs only one place to show them, and the user always receives a clear message. The most useful design decision was to keep the data logic separate from the interface: the screen is only a view of the plan dictionary, so the same functions serve the screen, the files and the reports.

The results depend on the information entered. The budget is based on estimates of the daily cost, and the program does not know real prices. The test scenarios in Table 5 should be repeated on the computer where the project is demonstrated.

13. Advantages

- A single, simple window for destinations, bookings, expenses and preferences.
- Automatic budget calculation with a warning when the limit is exceeded.
- Works offline, needs no account and no installation beyond Python.
- The plan is kept between sessions and can be exported to CSV and text files.
- Friendly messages for wrong input and bad files.
- Clear separation of data logic, interface and storage, which makes the code easy to read, test and extend.

14. Limitations and Security Considerations

- The plan is stored with `pickle`. A pickle file from an unknown source can run code when it is opened, so only files created by this program should be loaded.
- Plan files are not encrypted or protected by a password.
- The program is meant for one user on one computer and does not share or merge plans.
- The budget uses the amounts typed by the user. There are no live prices and no currency conversion, so all amounts must be in one currency.
- There is no map, weather or online booking, and no PDF export.
- Regular backups of the plan files are advisable.

15. Future Enhancements

- Store the plan in JSON instead of pickle, which is safer and can be read in any text editor.
- Use SQLite or a cloud database for searching, larger data and sharing.
- Add maps and weather through online services.
- Add currency conversion for trips abroad.
- Export the itinerary and the budget report to PDF.
- Allow several users to share and edit one plan.
- Add automated tests for the logic functions.

16. Conclusion

Travel Planner turns several separate topics of the syllabus into one useful application. Dictionaries and lists hold the trip, functions calculate the budget and the itinerary, file handling keeps the plan between sessions, and exception handling makes the program dependable. Keeping the logic independent of the Tkinter interface made the code simpler and easier to test. The project can be extended with a safer file format, a database, online services and sharing between users.

17. References

[1] Python Software Foundation, “tkinter — Python interface to Tcl/Tk”. https://docs.python.org/3/library/tkinter.html

[2] Python Software Foundation, “tkinter.ttk — Tk themed widgets”. https://docs.python.org/3/library/tkinter.ttk.html

[3] Python Software Foundation, “pickle — Python object serialization”. https://docs.python.org/3/library/pickle.html

[4] Python Software Foundation, “csv — CSV File Reading and Writing”. https://docs.python.org/3/library/csv.html

[5] Python Software Foundation, “Errors and Exceptions”. https://docs.python.org/3/tutorial/errors.html

[6] Python Software Foundation, “Data Structures”. https://docs.python.org/3/tutorial/datastructures.html